



**acm** International Collegiate  
Programming Contest

**IBM**

event  
sponsor

# 函数库

Ciallo~( $\angle \cdot \omega <$ ) $\frown$ ☆



# 目 录

|                 |    |
|-----------------|----|
| 函数库 .....       | 5  |
| 关于STL .....     | 5  |
| 数据转义 .....      | 5  |
| 字符串元素统计 .....   | 5  |
| 连续输入 .....      | 6  |
| swap .....      | 6  |
| iter_swap ..... | 6  |
| cstring .....   | 7  |
| 常用函数 .....      | 7  |
| to_string ..... | 8  |
| memset .....    | 8  |
| getline .....   | 9  |
| find .....      | 9  |
| compare .....   | 10 |
| replace .....   | 11 |
| cmath .....     | 11 |
| 常用函数 .....      | 11 |
| 指数 .....        | 12 |
| pow .....       | 12 |
| exp .....       | 12 |
| 对数 .....        | 12 |
| log .....       | 12 |
| 开根 .....        | 13 |
| sqrt .....      | 13 |
| cbrt .....      | 13 |
| 绝对值 .....       | 13 |
| abs .....       | 13 |
| 取整 .....        | 13 |
| ceil .....      | 13 |
| floor .....     | 14 |
| round .....     | 14 |
| 三角函数 .....      | 14 |

|                         |    |
|-------------------------|----|
| algorithm .....         | 14 |
| 常用函数 .....              | 15 |
| sort .....              | 15 |
| sort .....              | 15 |
| stabl_sort .....        | 17 |
| partial_sort .....      | 17 |
| partial_sort_copy ..... | 17 |
| is_sorted .....         | 17 |
| reverse .....           | 18 |
| unique .....            | 18 |
| count .....             | 18 |
| count_if .....          | 19 |
| lower_bound .....       | 19 |
| next_permutation .....  | 20 |
| nth_element .....       | 21 |
| minmax_element .....    | 22 |
| minmax .....            | 22 |
| 仅GNU .....              | 23 |
| map .....               | 23 |
| 常用函数 .....              | 24 |
| 定义 .....                | 24 |
| 插入元素 .....              | 24 |
| 访问元素 .....              | 25 |
| unordered_map .....     | 26 |
| 重载 .....                | 27 |
| vector .....            | 27 |
| 常用函数 .....              | 27 |
| 定义及初始化 .....            | 28 |
| 增删特定位置元素 .....          | 28 |
| 删除指定数值元素 .....          | 30 |
| 查找指定数值的元素 .....         | 30 |
| 遍历元素 .....              | 31 |
| 动态二维数组 .....            | 31 |

|                     |    |
|---------------------|----|
| queue .....         | 32 |
| 常用函数 .....          | 33 |
| 定义及初始化 .....        | 33 |
| 清空 .....            | 33 |
| 元素访问 .....          | 34 |
| 优先队列 .....          | 34 |
| deque .....         | 35 |
| 常用函数 .....          | 35 |
| 定义及初始化 .....        | 36 |
| list .....          | 36 |
| 常用函数 .....          | 37 |
| 遍历链表 .....          | 37 |
| stack .....         | 38 |
| 常用函数 .....          | 38 |
| 清空 .....            | 38 |
| 例 .....             | 38 |
| set .....           | 39 |
| 常用函数 .....          | 39 |
| 例 .....             | 40 |
| 重载 .....            | 40 |
| multiset .....      | 41 |
| unordered_set ..... | 41 |
| bitset .....        | 41 |
| 常用函数 .....          | 41 |
| 初始化 .....           | 42 |
| iomanip .....       | 43 |
| 常用函数 .....          | 43 |
| setprecision .....  | 43 |
| setw .....          | 44 |
| 对齐 .....            | 44 |
| setfill .....       | 45 |
| numeric .....       | 45 |
| 常用函数 .....          | 45 |
| accumulate .....    | 45 |

|                           |    |
|---------------------------|----|
| partial_sum .....         | 45 |
| adjacent_difference ..... | 46 |
| iota .....                | 46 |
| sstream .....             | 46 |
| 定义及初始化 .....              | 46 |
| 例 .....                   | 47 |
| tuple .....               | 48 |
| 声明 .....                  | 48 |
| 元素访问 .....                | 48 |
| tie .....                 | 49 |
| ctime .....               | 49 |
| clock_t .....             | 49 |
| 扩展库ext .....              | 50 |
| 关于pb_ds .....             | 51 |
| hash .....                | 51 |
| 定义 .....                  | 51 |
| 例 .....                   | 52 |
| tree .....                | 52 |
| 定义 .....                  | 52 |
| 常用函数 .....                | 53 |
| 例 .....                   | 53 |
| 堆 .....                   | 54 |
| 定义 .....                  | 55 |
| 常用函数 .....                | 55 |
| 例 .....                   | 56 |
| trie .....                | 58 |
| 定义 .....                  | 58 |
| 常用函数 .....                | 58 |
| 例 .....                   | 58 |
| rope .....                | 59 |
| 定义 .....                  | 60 |
| 常用函数 .....                | 60 |
| 例 .....                   | 60 |

# 函数库

## 关于STL

参考手册: [cppreference.com](http://cppreference.com)

[GCC, the GNU Compiler Collection - GNU Project](http://gcc.gnu.org/)

STL 即标准模板库 (Standard Template Library), 是 C++ 标准库的一部分, 里面包含了一些模板化的通用的数据结构和算法。由于其模板化的特点, 它能够兼容自定义的数据类型, 避免大量的造轮子工作。NOI 和 ICPC 赛事都支持 STL 库的使用, 因此合理利用 STL 可以避免编写无用算法, 并且充分利用编译器对模板库优化提高效率。

```
1 //c++万能头
2 #include <bits/stdc++.h>
```

## 数据转义

```
1 //整型转义 0b二进制 0x十六进制
2 int n = 0b101;//n = 5;
3 int inf = 0x3f'3f'3f'3f;//可以插入 ' 作为分隔符, 方便阅读而不影响编译
```

```
1 //字符串取消转义 c++11 语法为string s = R"(.....)"; 括号内为字符串
2 string s = R"(hello\nworld)";//直接输出 hello\nworld, 而不会将\n转义为换行
```

## 字符串元素统计

```
1 //c++风格字符串
2 string a = "ab c\0de";
3 cout << a.size() << endl; //4 统计到\0(或输入按回车)
4 cout << a.length() << endl; //4 统计到\0(或输入按回车)
```

```
1 //c语言风格字符串
2 char s[999] = "ab c\0de";
3 cout << strlen(s); //4
```

## 连续输入

以回车停止

```
1 int arr[1005];
2 do { cin >> arr[i];i++; } while (cin.get() != '\n'); //用do..while否则会吞掉第一个输入
```

以EOF停止

```
1 //c++
2 int n;
3 while (cin >>n) {cout << n << endl; } //以EOF结束
4 while (cin >>n&&n != -1) {cout << n << endl; } //以输入-1结束
5
6 //c语言
7 int n;
8 while(scanf("%d",&n) != EOF) { printf("%d",n);} //以EOF结束
```

## swap

交换两个元素

```
1 //交换两个值
2 int a = 10,b = 20;
3 swap(a,b);//交换后a = 20, b = 10
```

```
1 //交换两个容器
2 vector<int>a = {1,2,3},b = {4,5};
3 swap(a,b);//交换后a = {4,5} b = {1,2,3}
```

## iter\_swap

`iter_swap(it1,it2)` 用于交换两个迭代器指向的值。

诺使用 `swap(it1,it2)`，只是交换了两个“指向节点的指针”（即迭代器），而不是交换原容器中的节点。

```
1 list<int>q = {1,2};
```



```

2 map<int, list<int>::iterator> mp;
3 mp[1] = q.begin();
4 mp[2] = next(q.begin(), 1);
5
6 auto it = q.begin();
7 auto it2 = next(it);
8
9 for(auto x:q){
10     cout << x << ' ';
11 }cout << endl;
12
13 iter_swap(it, it2); //或者使用swap(*it, *it2), 而非swap(it, it2)
14
15 for(auto x:q){
16     cout << x << ' ';
17 }cout << endl;
18
19 cout << *mp[1] << ' ' << *mp[2] << endl;

```

## cstring

### 常用函数

| 常用函数                                                    |                                                 |
|---------------------------------------------------------|-------------------------------------------------|
| <b>min(a,b)/max(a,b);</b>                               | 返回a,b最小/最大的数                                    |
| <b>a.push_back(x);</b>                                  | 在末尾添加一个元素x                                      |
| <b>a.pop_back(x);</b>                                   | 删除末尾元素x                                         |
| <b>a.insert(pos,x);</b>                                 | 在pos点插入元素x                                      |
| <b>s.erase(pos,len);</b>                                | 删除下标pos开始，长度为len的子串                             |
| <b>a.clear();</b>                                       | 清空元素                                            |
| <b>a.replace(pos,len,"abc");</b>                        | 元素替换                                            |
| <b>a.find("abc"[,pos])</b> [从下标pos位置开始查找,不填默认为0];       | 返回第一次出现目标字符串的位置,没有则返回string::npos(通常定义为-1或无穷大)  |
| <b>a.find_last_of("abc"[,pos])</b> [从下标pos位置开始,不填默认为0]; | 返回最后一次出现目标字符串的位置,没有则返回string::npos(通常定义为-1或无穷大) |

| 常用函数                                       |                                                          |
|--------------------------------------------|----------------------------------------------------------|
| <code>getline(cin, str, c);</code>         | 输入字符串str,包括空格和回车,遇到c(不写默认为'\n')停止                        |
| <code>str1.compare(str2);</code>           | 比较两个字符串的字典序                                              |
| <code>memset(arr, 'c', sizeof arr);</code> | 初始化char[]型字符串为字符'c',或将数组元素按字节初始化为0/-1/0x3f               |
| <code>a+=b;</code>                         | 字符串合并                                                    |
| <code>a.size(); a.length();</code>         | 返回字符串元素个数                                                |
| <code>a.empty();</code>                    | 返回字符串是否为空                                                |
| <code>count(a.begin(), a.end(), c);</code> | 计数字符c出现的次数                                               |
| <code>a.substr(pos, len);</code>           | 返回下标从pos开始长度为len的子串 (若pos越界则会异常报错,若pos+len越界则只会拷贝到字符串末尾) |
| <code>a.c_str();</code>                    | 返回这个string对应的字符数组的头指针                                    |

## to\_string

数字转字符串

```
1 int n = 114514;
2 string a = to_string(n);
```

```
1 //字符串转数字
2 string a = "114514";
3 int n = atoi(a.c_str()); //从第一位开始,遇到非数字停止,若第一位不是数字则为0
```

## memset

#include

按字节初始化

```
1 //char[]型字符串
2 char arr[100];
3 memset(arr, 'a', sizeof arr); //可以初始化为任意字符
4 for (int i = 0; i < 100; i++) cout << arr[i];
```

```

1 //非字符型数组
2 int arr[100]; //按字节初始化, int为4字节
3 memset(arr, -1, sizeof arr); //一般初始化为-1、0、0x3f或-0x3f
4 //-1 => (1111 1111 1111 1111) = -1
5 //0 = (0000 0000 0000 0000) = 0
6 //0x3f = (3f 3f 3f 3f) = 0x3f3f3f3f = 1061109567 > 1e9
7 for (int i = 0; i < 100; i++) cout << arr[i];

```

对一个浮点类型的数memset(a,-1,sizeof a)元素会初始化为 nan(not a number), 与任何比较大小的结果都为false

## getline

带空格/回车输入字符串

有时需要在每组**第一次**使用前加getchar();缓冲, 防止读入上一个数据的回车。

```

1 string str;
2 getline(cin, str, c); //读取到字符c则停止, 诸参数c不写, 则默认读取到回车停止, c可以为('字符'),
   ('\\n'), (' ') 等
3 cout << str;

```

## find

rfind则为逆向查找, 从字符串后面往前查找

当find函数未找到要查找的子字符串时, 它将返回常量值string::npos 通常定义为-1; 或一个很大的无符号整数类型的值

```

1 //查找目标字符串的下标
2 #include<iostream>
3 #include<string>
4 using namespace std;
5 int main() {
6     string arr = "komeijikoishi";
7
8     string target = "oi";
9     int index = arr.find(target); //返回第一次出现目标字符串的位置
10    if(index == -1) cout << "不存在子串" << endl;
11    else cout << index << endl;
12
13    int index2 = arr.find("i", 5); //从下标为5的位置开始查找
14    cout << index2 << endl;
15

```

```

16
17     int index3 = arr.find_first_of("i");    //查找目标字符串第一次出现的位置
18     cout << index3 << endl;
19     int index4 = arr.find_last_of("i");    //查找目标字符串最后一次出现的位置
20     cout << index4 << endl;
21
22     return 0;
23 }

```

```

1 //查找目标字符串出现的中次数
2 string s, c;
3 while (cin >> s >> c) {
4     int index = 0; //用来存储不断更新最新找到的位置
5     int sum = 0; //累加出现的次数
6     while ((index = s.find(c, index)) != string::npos) {
7         cout << "sum: " << sum+1 << " index: " << index << endl;
8         index += c.length(); //上一次s中与c完全匹配的字符应跳过，不再比较
9         sum++;
10    }
11    cout << sum << endl;    // 11111111中11出现了3次
12 }

```

```

1 //查找字符串中，某一个字符出现的次数
2 string a = "abcdefga";
3 cout << count(a.begin(), a.end(), 'a');

```

## compare

比较字典序

区分大小写，从头开始依次比较字符对应ASCII码大小

```

1 string A = "12345";
2 string B = "12454";
3 cout << A.compare(B);
4 //A>B 返回1;   A<B 返回-1;   A==B 返回0;

```

```

1 //str_cmp比较char类型字符串的字典序
2 char A[] = "abc";
3 char B[] = "abc";
4 strcmp(A,B)
5 //A>B 返回1;   A<B 返回-1;   A==B 返回0;

```

# replace

## 字符替换

```
1 //a.replace(pos,len,"str"); 从pos位置开始, 长度为len, 替换为str
2 string a = "abcdefg";
3 a.replace(0, 4, "66");//或者用迭代器a.replace(a.begin(), a.begin() + 4, "66");
4 //replace(a.begin(),a.end(),'a','*');
5 cout << a;//66efg
6
7 //a.replace(pos,len,n,'c'); 从pos位置开始, 长度为len, 替换为重复n次的字符c
8 string a = "abcdefg";
9 a.replace(0,4,3,'6');
```

```
1 //将字符串中的a用b替换
2 replace(str.begin(), str.end(), 'a', 'b'); //需包含#include <algorithm>
```

# cmath

## 常用函数

| 常用函数                            |                                 |
|---------------------------------|---------------------------------|
| pow(a,b)                        | a^b次方                           |
| sqrt(n)                         | sqrt开平方根 cbrt开立方                |
| abs(n)                          | 绝对值                             |
| ceil(n), floor(n), round(n)     | 向上取整, 向下取整, 四舍五入取整              |
| round(n)                        | 四舍五入到整数                         |
| log2(n), log10(n), log(n)       | 求以2/10/e为底的对数,logf、logl为单精度、高精度 |
| sin, cos, tan, asin, acos, atan | 三角函数                            |

## 指数

### pow

`n * pow(a,b);` //相当于 $n * (a^b)$

```
1 //pow参数类型和返回值类型均为浮点型，可能会导致输出与预期不符而wrong answer
2 int a = 1234;
3 cout << a*a << endl; //1522756
4 cout << pow(a, 2) << endl; //1.52276e+06
```

可以用 `int n = pow(a,2)` 类型转换再输出n

### exp

`exp(x);` 返回  $e^x$ ，在 `double` 内，x 的有效区间为  $[-708.4, 709.8]$

## 对数

### log

```
1 log(x);
```

| 函数                    |                    |
|-----------------------|--------------------|
| <code>log(x)</code>   | $\ln x$ ，以 $e$ 为底  |
| <code>log10(x)</code> | $\lg x$ ，以 10 为底   |
| <code>log2(x)</code>  | $\log_2 x$ ，以 2 为底 |

## 开根

### sqrt

```
1 sqrt(n);    //开平方根,参数为double
2 sqrtf(n);   //参数为float
3 sqrtl(n);   //高精度,参数类型是long double;防止long long类型强转为double的误差
4
5
6 pow(n,1.0/m); //开n次方 (1.0/m或1/m.0)
```

### cbrt

```
1 cbrt(n);    //开三次方
```

## 绝对值

请使用C++的 `std::abs`, 而非C的 `abs`, 后者只针对 `int` 类型, 容易造成计算错误

### abs

```
1 abs(n);
2 fabs(n);
3 fabsl(n);
```

## 取整

### ceil

```
1 double n = 1.5;
2 //向上取整 ceil(1.5) = 2;
3 ceil(n) = 2;
4 ceil(2/3) = 0; //相当于ceil(0)
5 ceil(1.0*2/3) = 1; //整型相除应先转换为浮点型, 再取整
6
```

## floor

```
1 double n = 1.5;
2 //向下取整 floor(1.5) = 1;
3 floor(n) = 1;
```

## round

```
1 double n = 1.5;
2 //四舍五入取整
3 round(n); //取最接近整数(中间情况下远离0取整)
4 printf("%.2lf", n); //四舍五入保留2位小数, long double类型改为%.2Lf
5 round(n*100)/100; //四舍五入保留2位小数
```

## 三角函数

```
1 sin(x);
2 cos(x);
3 tan(x);
4
5 asin(x);
6 acos(x); //acos(-1) =  $\pi$ 
7 atan(x); //取值范围( $-\pi/2, \pi/2$ )
```

```
1 atan2(y, x); //返回原点至(y, x)的方位角(即与x轴的夹角), 取值范围为( $-\pi, \pi$ ]。
2 //若要以度数表示, 将结果*180/ $\pi$ 即可
3 //比atan稳定。当y远大于x时, atan计算结果是不稳定的;
```

---

## algorithm



# 常用函数

| 常用函数                            |                                                                  |
|---------------------------------|------------------------------------------------------------------|
| sort(begin,end,cmp)             | cmp为空则默认升序排序                                                     |
| is_sorted(begin,end)            | 返回所给序列是否已按排列(默认判断是否非递减)                                          |
| lower_bound(begin,end,value)    | 返回第一个>=v值的迭代器(-begin()即为下标)                                      |
| upper_bound(begin,end,value)    | 返回一个>v值的迭代器(-begin()即为下标)                                        |
| count(begin,end,value)          | 计数v值出现了多少次                                                       |
| count_if(begin,end,cmp)         | 按自定义cmp规则计数                                                      |
| reverse(begin,end)              | 翻转所在区间数值                                                         |
| find(begin,end,value)           | 返回v值第一次出现的迭代器,没有则返回指向容器末尾的迭代器                                    |
| erase(begin,end)                | 删除范围内元素                                                          |
| unique(begin,end)               | [移除范围内的连续重复元素,返回去重之后的尾地址(使用前先排序)][实际并没有删除,只是将重复元素放到容器末尾,不改变容器大小] |
| next_permutation(begin,end)     | 将元素排列为下一个字典序,没有下一个则返回0, prev_permutation上一个字典序                   |
| nth_element(begin,nth,end)      | 重排first~last中元素使第nth小的数在第nth位置,左边的数都小于它,右边的数都大于它                 |
| max_element(begin,end)          | 返回所给范围内最大值的迭代器 min_element类似                                     |
| minmax({n1,n2,n3,...})          | 返回参数中的最小值和最大值, 返回类型为pair<min,max>                                |
| fill(begin,end,value)           | 将范围内元素赋值为value                                                   |
| search(begin1,end1,begin2,end2) | 返回begin1~end1中第一次出现begin2 ~end2中的迭代器,没有则返回指向end1的迭代器             |

## sort

### sort

语法：sort(begin, end, cmp);

其中begin为指向待sort()的数组的 第一个元素的指针，end为指向待sort()的数组的 最后一个元素的下一个位置的指针，cmp参数为排序准则(不写的话，默认从小到大进行排序);

时间复杂度：n\*log2(n)

```

1 //从小到大
2 sort(arr, arr+10);
3 sort(str.begin(),str.end());
4 //从大到小则cmp = greater<int>()
5 sort(arr, arr+10, greater<int>());
6 sort(str.begin(),str.end(),greater<int>());
7 //或者sort(str.rbegin(),str.rend());//sort(a.rbegin(),a.rend()-1)//下标从1开始

```

## 自定义cmp排序准则

需注意: `sort` 比较两个结构体, 不能出现 `a < b` 和 `b < a` 同时为真的情况, 否则会运行错误

```

1 //结构体排序
2 #include <iostream>
3 #include <algorithm>
4 using namespace std;
5 struct hero{ string name;int sco; };
6 void info(hero arr[], int len){
7     string nameSeed[5] = {"小毛","小刘","小李","小张","小王"};
8     for (int i = 0;i < len; i++) {
9         arr[i].name += nameSeed[i];
10        cout << "姓名: " << arr[i].name;
11        cout << "\t请输入分数: ";cin >> arr[i].sco;
12    }
13 }
14
15 bool agerank(hero a, hero b) {
16     return a.sco > b.sco;
17     //自定义排序, 以结构体sco大小为依据
18 }
19
20 void printInfo(hero arr[],int len) {
21     cout << "\n\n排序后\n" << endl;
22     for (int i = 0;i < len;i++) {
23         cout << "姓名: " << arr[i].name << "\t分数: " << arr[i].sco << endl;
24     }
25 }
26 int main() {
27     hero arr[5];
28     int len = sizeof(arr) / sizeof(arr[0]);
29     info(arr, len);//1.输入信息
30     sort(arr, arr+len,agerank);//2.排序
31     printInfo(arr, len);//3.输出
32 }

```

## stabl\_sort

稳定排序，但需要额外的空间开销

## partial\_sort

将区间最小的前k个按升序排在前面

时间复杂度 $O(N \cdot \log M)$  其中N为区间长度,M为前M个长度

`partial_sort(first, first+k, last);`

`partial_sort(first, first+k, last, cmp);` //cmp不写默认升序

```
1 | vector<int>a = {6,9,2,3,1,5,7,5,4};
2 | partial_sort(a.begin(),a.begin()+5,a.end());
3 | //a = 1 2 3 4 5 9 7 6 5
```

## partial\_sort\_copy

不改变原来的容器，而是先将选定的部分元素拷贝到另外指定的容器中，然后再对这部分元素进行排序。

`partial_sort_copy(first1, last1, first2, last2);`

`partial_sort_copy(first1, last1, first2, last2, cmp);`

```
1 | vector<int>a = {6,9,2,3,1,5,7,5,4},b(5);//若b的大小不足5个,则会舍去后面的
2 | partial_sort_copy(a.begin(),a.end(),b.begin(),b.begin()+5);//b = 1 2 3 4 5
```

## is\_sorted

返回值为bool类型

`is_sorted(first,end);`

判断区间元素是否已经排序,默认判断升序

`is_sorted(first,end,cmp)`

```
1 | is_sorted(a.begin(),a.end(),[](auto &e1,auto &e2){return e1 > e2;})//是否非递增
2 | is_sorted(a.begin(),a.end(),[](auto &e1,auto &e2){return e1 >= e2;})//是否严格
   | 递减
```

## reverse

reverse(begin,end) 翻转范围内的数

```
1 reverse(str.begin(), str.end());    //字符串倒序
```

```
1 reverse(a,a+n);    //数组的翻转，n为数组元素个数
```

## unique

[移除范围内的连续重复元素,返回去重之后的尾部迭代器(使用前先排序)][实际并没有删除,只是将重复元素放到容器末尾,不改变容器大小]

```
1 unique(v.begin()+1, v.end());
2 unique(a+1,a+n+1);
3 //int m = unique(a+1,a+n+1) - a - 1;
4 //unique前: 2 3 3 4 4 5 8 9 9
5 //unique后: 2 3 4 5 8 9 8 9 9
6 //a[1~m] = {2 3 4 5 8 9}
```

```
1 //配合erase删除 适用于容器如 vector
2 erase(unique(v.begin(),v.end()),v.end());
```

## count

在序列中统计某个值出现的次数

```
cout<<count(arr.begin(), arr.end(), searchValue) << endl;
```

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     int n,x, arr[100]; cin >> n>>x;
5     for (int i = 0;i < n;i++)
6         cin >> arr[i];
7     cout << count(arr,arr+n,x);
8 }
```

## count\_if

返回满足条件范围内的元素个数

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4  using namespace std;
5  vector<int> myvector;
6  bool isEven(int elem){ return elem % 2 == 0;}
7  bool cmp(int elem) { return elem > 2; }
8  void main(){
9      for (int i = 0;i < 9;i++){
10         myvector.push_back(i+1);
11         cout << myvector[i];
12     }
13     // 计算偶数个数
14     int ctif = count_if(myvector.begin(), myvector.end(), isEven);
15     // 计算大于2的个数
16     int ctg = count_if(myvector.begin(), myvector.end(), cmp);
17     cout << "偶数元素个数:" << ctif << endl;
18     cout << "超过2的元素个数: " << ctg << endl;
19 }
```

## lower\_bound

使用时需要保证数组为有序非递减

lower\_bound( begin,end,num): 从数组的begin位置到end-1位置二分查找第一个**大于或等于num的数字**，找到返回该数字的地址，不存在则返回end。通过返回的地址减去起始地址begin,得到找到数字在数组中的下标。

upper\_bound( begin,end,num): 从数组的begin位置到end-1位置二分查找第一个**大于num的数字**，找到返回该数字的地址，不存在则返回end。通过返回的地址减去起始地址begin,得到找到数字在数组中的下标。

```
1  #include <iostream>
2  #include <vector>
3  #include <iomanip>
4  using namespace std;
5
6  vector<int>v;
7
8  int main() {
9      for (int i = 1; i <= 9;i++) {
10         v.push_back(i * 2);
11         v.push_back(i * 2);
12         v.push_back(i * 2);
13     }
```

```

14     for(auto& i:v){
15         cout << i << ' ';
16     }puts("");
17
18     int x; cin >> x;
19
20     int it1 = lower_bound(begin(v), end(v), x) - begin(v); //尽量不要用auto
21     int it2 = upper_bound(begin(v), end(v), x) - begin(v);
22     if (it1 == size(v)) cout << -1 << " ";
23     else cout << it1 << " ";
24
25     if (it2 == size(v)) cout << -1 << endl;
26     else cout << it << endl;
27
28     return 0;
29 }

```

```

1 // 例题:https://www.luogu.com.cn/problem/P2249
2 #include<iostream>
3 using namespace std;
4 int a[1000005];
5 int main() {
6     int n, m;
7     scanf("%d%d", &n, &m);
8     for(int i = 1; i<=n; i++) {
9         scanf("%d", &a[i]);
10    }
11    while(m--) {
12        int x;
13        scanf("%d", &x);
14
15        int tot = lower_bound(a+1, a+n+1, x) - a; //返回第一个大于等于x的位置
16        int ans = upper_bound(a+1, a+n+1, x) - a; //返回第一个大于x的位置
17        printf("%d\n", tot);
18    }
19
20    return 0;
21 }

```

## next\_permutation

prev\_permutation求当前排列的上一个排列

求当前排列的下一个排列，当当前序列不存在下一个排列时，函数返回false，否则返回true。

```

1 //全排列需要先对数组按升序排序
2 int arr[100];
3 int n; cin >> n;
4 for (int i = 0; i < n; i++)
5     cin >> arr[i];
6
7 sort(arr, arr + n);
8
9 do {for (int i = 0; i < n; i++){
10     cout << arr[i] << ' ';
11     }cout << endl;
12 }
13 while (next_permutation(arr, arr + n));

```

```

1 //prev_permutation求当前排列的上一个排列
2 string a = "cadb";
3 sort(a.begin(), a.end(), greater<int>());
4 do {cout << a << endl;
5 } while (prev_permutation(a.begin(), a.end()));

```

## nth\_element

时间复杂度 $O(N)$

`nth_element(first, nth, last)`会将重排`first~last`中元素使得小于 `*nth` 的在其左边，大于等于 `*nth` 的在其右边

```

1 //求第k小数
2 #include <iostream>
3 #include <algorithm>
4 using namespace std;
5 const int N = 5000006;
6 int a[N];
7
8 int main(){
9     int n, m; cin >> n >> m;
10    for(int i = 0; i < n; i++) scanf("%d", &a[i]);
11
12    nth_element(a, a+m, a+n);
13
14    cout << a[m];
15 }

```

## minmax\_element

`minmax_element(first, last)`

返回区间最小值和最大值的迭代器，返回类型为`pair<min,max>`

```
1 auto ans = minmax_element(a.begin(), a.end());
2 cout << *ans.first << ' ' << *ans.second << endl;
```

```
1 auto [mn, mx] = minmax_element(a.begin(), a.end());
2 cout << *mn << ' ' << *mx << endl;
```

`max_element(first, last)`   `min_element(first, last)`

返回区间 最大值 或 最小值 的迭代器

```
1 int mx = *max_element(a.begin(), a.end());
2 int mn = *min_element(a.begin(), a.end());
3 cout << mx << endl << mn << endl;
```

## minmax

元素个数大于两个要打花括号

`minmax({n1,n2,n3, ... })`

返回最小的和最大的元素，返回类型为`pair(min,max)`

```
1 auto [mn, mx] = minmax({n1, n2, n3, n4}); //pair<int,int> ans =
    min({n1, n2, n3, n4});
2 cout << mn << ' ' << mx << endl;
```

`min({n1,n2,n3, ... })`

返回最小的元素，max类似

```
1 int mn = min({a1, a2, a3, a4});
2 int mx = max({a1, a2, a3, a4});
3 cout << mn << ' ' << mx << endl;
```



## 仅GNU

仅在GNU编译器可用，否则会编译错误

```
1 | __lg(x)
```

- 返回  $\lfloor \log_2 x \rfloor$ 。
- 时间复杂度  $O(1)$ 。
- 常用于实现倍增、二进制运算等。

```
1 | __gcd(x, y)
```

- 返回  $\gcd(x, y)$ 。
- 复杂度是对数级别的，常数较小。
- 注意，返回值的符号 **不一定** 是正。
- 在 C++17 之前都是很常用的。

```
1 | __builtin_popcount(x)    //返回 x 在二进制下 1 的个数
2 | __builtin_parity(x)     //返回 x 在二进制下 1 的个数的奇偶性，略快于
   | __builtin_popcount(x)&1
3 | __builtin_ffs(x)        //返回二进制下最后一个 1 是从右往左第几位(从低位到高位)
4 | __builtin_ctz(x)        //返回二进制下后导零的个数，x=0 时未定义
5 | __builtin_clz(x)        //返回二进制下前导零的个数，x=0 时未定义
```

**复杂度约为  $O(\log \log n)$  且常数很小，一般比手写位运算快。**

如果  $x$  的类型是 `long long`，请务必使用 `__builtin_xxxll(x)`

---

## map

## 常用函数

| 常用函数             |                                                 |
|------------------|-------------------------------------------------|
| m.begin()        | 返回指向map头部的迭代器                                   |
| m.end()          | 返回指向map末尾的迭代器                                   |
| m.insert({x,y})  | 插入元素(pair类型)                                    |
| m.clear()        | 删除所有元素                                          |
| m.size()         | 返回map中元素的个数                                     |
| swap(m1,m2)      | 交换两个map容器的所有元素                                  |
| m.empty()        | 返回容器是否为空                                        |
| m.erase()        | 参数为一个迭代器或键值则删除该元素，参数为两个迭代器时则删除两个迭代器之间的元素O(logN) |
| m.find(x)        | 返回键值为x的元素的迭代器,不存在则返回end()                       |
| mp.lower_bound() | 返回一个迭代器，指向键值 >= x 的第一个元素                        |
| mp.upper_bound() | 返回一个迭代器，指向键值 > x 的第一个元素                         |

## 定义

map内部是自动排序的

```
map<type1name,type2name> m1;//第一个是关键字key(每个关键字只能出现一次)的类型，第二个是对应的值value的类型
```

数据类型可以是string, char, int, iterator, pair等多种类型

```
multimap<type1name,type2name> m2;可以有多个相同的键值对
```

## 插入元素

```
map<int, string> student;
```

```
1 //1.用insert函数插入pair数据
2 student.insert(pair<int, string>(000, "student_zero"));
```

```
1 //2.用insert函数插入value_type数据
2 student.insert(map<int, string>::value_type(001, "student_one"));
```

```

1 //3.用"array"方式插入
2 mapStudent[1] = "student_first";
3 mapStudent[2] = "student_second";

```

## 访问元素

若不确定一个元素是否存在，避免使用 `[]` 查找，而是要用 `find()` 函数。因为用 `[]` 如果不存在会新建一个元素反复使用会造成 TLE。

```

1 //通过键值
2 #include <iostream>
3 #include <map>
4 using namespace std;
5 map<string,int>student;
6 int main() {
7     int n; cin >> n;
8     for (int i = 0; i < n; i++){
9         string name; int num;
10        cin >> name >> num;
11        student[name] = num;
12    }
13    //输入姓名 查找学号
14    string find_name; cin >> find_name;
15    cout << student[find_name] << endl;
16    return 0;
17 }

```

```

1 //for(auto &i: m) 遍历元素    c++11
2 #include <iostream>
3 #include <map>
4 using namespace std;
5 map<string, int>m;
6 int main() {
7     for (int i = 0; i < 3;i++) {
8         string s; cin >> s;
9         int n; cin >> n;
10        m[s] = n;
11    }
12    for (auto& i:m){
13        cout << i.first << ' ' << i.second << endl;
14    }
15    for(pair<const string,int>& i:m){//键值不可修改
16        cout << i.first << ' ' << i.second << endl;
17    }
18    for(auto& [x,y]:m){//c++17结构化绑定 //auto &[x,y] = *mp.begin();
19        cout << x < " " << y << endl;
20    }
21 }

```

```
22     return 0;
23 }
```

```
1  //通过迭代器遍历元素      map<string, int>::iterator it = student.begin();
2  #include <iostream>
3  #include <map>
4  using namespace std;
5  map<string, int> student;
6  int main() {
7      int n; cin >> n;
8      for (int i = 0; i < n; i++){
9          string name; int num;
10         cin >> name >> num;
11         student[name] = num;
12     }
13     //遍历姓名及学号
14     for (map<string, int>::iterator it = student.begin(); it != student.end();
15         it++)
16         cout << it->first << " " << it->second << endl;
17     //或
18     for (map<string, int>::iterator it = student.begin(); it != student.end();
19         it++)
20         cout << (*it).first << " " << (*it).second << endl;
21 }
```

## unordered\_map

```
#include <unordered_map>
```

unordered\_map 容器等价于无序的 map 容器。

无法使用pair类型作为key

unordered\_map通过相关的映射函数来加快寻找速度，因此，如果数据随机，unordered\_map的插入和查询速度接近 $O(1)$ 。但是由于hash冲突的存在，能被一些特意构造的数据卡到 $O(n)$ 的时间复杂度，当然，我们可以通过手写hash函数的形式来尽量规避此类问题。

## 重载

防止某些处心积虑的人卡 unordered\_map。

使用pair、结构体等类型作为umap/uset的key值时，需要手动重载哈希函数，因为它们没有默认的哈希函数。

源自CFblog更强的哈希函数（<https://codeforces.com/blog/entry/62393>）：

```
1  const auto RAND =
    std::chrono::steady_clock::now().time_since_epoch().count();  
2  struct hh {  
3      static int splitmix64(int x) {  
4          x += 0x9e3779b97f4a7c15;  
5          x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;  
6          x = (x ^ (x >> 27)) * 0x94d049bb133111eb;  
7          return x ^ (x >> 31);  
8      }  
9      int operator () (const int &x) const {  
10         return splitmix64(x + RAND);  
11     }  
12 };  
13  
14 unordered_map<int,int,hh>mp;  
15 //gp_hash_table<int,int,hh>hs; //或者pd_ds的hash_table
```

## vector

向量（Vector）是一个封装了动态大小数组的顺序容器（Sequence Container）。跟任意其它类型容器一样，它能够存放各种类型的对象。可以简单的认为，向量是一个能够存放任意类型的动态数组。

使用STL向函数传递容器时使用&引用传递或者\*指针传递，否则会发生拷贝构造，与数组不同

```
1  void function1(vector<int> vec)//传值      //值传递，会发拷贝构造，与数组不同  
2  void function2(vector<int>& vec)//传引用    //引用传递，不会发生拷贝构造  
3  void function3(vector<int>* vec)//传指针    //指针传递，不会发生拷贝构造
```

## 常用函数

| 常用函数           |            |
|----------------|------------|
| v.front/back() | 返回第一/最后个元素 |

| 常用函数                         |                       |
|------------------------------|-----------------------|
| v.push_back()/emplace_back() | 向容器末添加一个元素 O(1)       |
| v.pop_back()                 | 删除容器末的元素 O(1)         |
| v.insert()                   | 向容器中插入元素 O(n)         |
| v.erase(first,last)          | 删除指定位置元素 O(n)         |
| v.size()                     | 返回容器内元素个数             |
| v.empty()                    | 返回容器是否为空              |
| v.clear()                    | 清空容器                  |
| v1.compare(v2)               | 按字典序比较容器v1和v2         |
| v1.swap(v2)                  | 交换两个容器,vector特化版的swap |

```

1 //支持两个容器比较 按字典序比较
2 vector<int>a{1,3,3,4};
3 vector<int>b{1,2,16,5,5};
4 cout << (a > b) << endl;

```

## 定义及初始化

```

1 //不建议使用vector<bool>效率很低
2 //定义一个空的容器
3 vector<int>v1;
4 //定义一个空的vector类型数组
5 vector<int>arr[100];
6 //定义一个含有10个元素的容器
7 vector<int>v2(10); //int类型默认初始化为0
8 //定义一个含有10个元素的容器，并将其初始化
9 vector<string>v3(10,"abc");
10 //定义二维数组v[n+1][26]每个元素初始化为INF
11 vector<vector<int>>dp(n+1,vector<int>(26,INF));

```

## 增删特定位置元素

时间复杂度O(1)~O(N)，常数较小

```

1 A.push_back(x) ;//在末尾添加元素x  O(1)
2 A.pop_back(); //删除末尾元素      O(1)
3 A.insert(pos,x); //在迭代器pos位置添加元素x  O(n)
4 A.erase(first,last); //删除迭代器first到last之间的元素  O(n)
5 A.clear(); //清空所有元素 O(n)

```

//通过迭代器:

```

1 //insert(po,x) 在迭代器po位置插入元素x
2 vector<int> A = { 100,200,300,400,500,600 };
3 vector<int>::iterator it; //通过迭代器访问容器
4
5 it = A.begin();
6 A.insert(it, 111); //向索引为0的位置插入元素111
7 //输出内容为: 111 100 200 300 400 500 600
8 for (int i = 0; i < A.size(); i++) {
9     cout << A[i] << " ";
10 }
11 cout << endl;
12
13 it = A.begin() + 2;
14 A.insert(it, 222); //向索引为2的位置插入元素222
15 //输出内容为: 111 100 222 200 300 400 500 600
16 for (int i = 0; i < A.size(); i++) {
17     cout << A[i] << " ";
18 }
19 cout << endl;
20
21 it = A.end();
22 A.insert(it, 999); //向A的末尾插入元素999
23 //输出内容为: 111 100 222 200 300 400 500 600 999
24 for (int i = 0; i < A.size(); i++) {
25     cout << A[i] << " ";
26 }
27 cout << endl;

```

```

1 //erase() 函数来删除指定位置的元素
2 vector<int> A = { 100,200,300,400,500,600 };
3 vector<int>::iterator it;
4
5 it = A.begin(); //索引为0的位置
6 A.erase(it); //删除索引为0的位置的元素
7 //输出内容为: 200 300 400 500 600
8 for (int i = 0; i < A.size(); i++) {
9     cout << A[i] << " ";
10 }
11 cout << endl;
12
13 it = A.begin() + 2; //索引为2的位置
14 A.erase(it); //删除索引为2的位置的元素

```

```

15 //输出内容为: 200 300 500 600
16 for (int i = 0; i < A.size(); i++) {
17     cout << A[i] << " ";
18 }
19 cout << endl;

```

## 删除指定数值元素

```

1 //如果能在目标vector中找到该数值的元素，直接删除
2 //如果目标vector中找不到该数值的元素，不做任何操作，不会报错
3 vector<int> A = { 100,200,300,400,500,600 };
4 A.erase(remove(A.begin(), A.end(), 500), A.end()); //删除数值为500的元素
5 A.erase(remove(A.begin(), A.end(), 700), A.end()); //删除数值为700的元素
6 //输出内容为: 100 200 300 400 600
7 for (int i = 0; i < A.size(); i++) {
8     cout << A[i] << " ";
9 }
10 cout << endl;

```

```

1 //排序再去重
2 sort(alls.begin(), alls.end());
3 alls.erase(unique(alls.begin(), alls.end()), alls.end());
4 //unique把所有重复元素放到容器后面，再把不重复元素提前，返回最后一个不重复元素位置
  (algorithm)
5 //erase删除最后一个不重复元素位置到数组末尾的位置的所有元素（vector）

```

## 查找指定数值的元素

```

1 //使用 find() 函数查找 函数的返回值为迭代器或指针
2 vector<int> myVector = { 100,200,300,400,500,600 };
3 vector<int>::iterator it = find(myVector.begin(), myVector.end(), 500);
4 //输出内容为: 目标元素的索引为: 4
5 if (it != myVector.end()) {
6     cout << "目标元素的索引为: " << distance(myVector.begin(), it) <<endl;
7 }
8 else {
9     cout << "没有找到" <<endl;
10 }

```



## 遍历元素

vector v = { 100,200,300,400,500,600 };

```
1 //通过迭代器
2 for (vector<int>::iterator i2 = v.begin(); i2 != v.end();i2++) {
3     cout << *i2 << endl;/**i2解引用
4 }
5
6 //或者这里直接使用auto, 不需要根据myVector中元素的类型改变, 自动推断变量类型
7 for (auto it : v) {/**修改it时只会影响副本元素, 不会修改原容器v的元素
8     cout << it << " ";
9 }
10 for (auto &it : v){/**&引用可以修改原容器v的元素
11     cout << it << " ";
12 }
```

```
1 //通过下标
2 for (int i = 0;i < v.size();i++){
3     cout << v[i] << " ";
4 }
```

## 动态二维数组

例如当 $1 \leq n, m \leq 1e6$ , 而 $n * m \leq 1e6$ ;  
无法直接构建arr[n][m], 需要使用动态数组

```
1 #include <iostream>
2 #include <vector>
3 #include <iomanip>
4 using namespace std;
5 const int N = 100005;
6 vector<int>v[N];/**v[0]={}, v[1]={}, v[2]={}...
7
8 int main() {
9     int n = 9;
10    for (int i = 1; i <= n; i++) {
11        v[i].emplace_back(0);/**如果需要让数组下表从1开始, 则需要填充第一位
12        for (int j = 1; j <= n; j++) {
13            v[i].emplace_back(i*j);
14        }
15    }
16
17    for (int i = 1; i <= n; i++) {
18        for (int j = 1; j <= n; j++) {
19            cout << left << setw(3)<< v[i][j] << ' ';
20        }puts("");
21    }
```

```

22
23
24     return 0;
25 }

```

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5  vector<vector<int>>>v;//V = {v1,v2,v3...}
6  vector<vector<int>>>dp(n+1,vector<int>(26,INF));//v[n+1][26]每个元素初始化为INF
7  //vector dp(n + 1, vector<int>(n + 1, 1e9)); //c++17
8  int main() {
9      //构建一个arr[n][m];的动态二维数组
10     int n, m; cin >> n >> m;
11     for (int i = 0; i < n; i++) {
12         vector<int>v1;
13         for (int j = 0; j < m; j++) {
14             int x; cin >> x;
15             v1.push_back(x);
16         }
17         v.push_back(v1);
18     }
19
20     for (int i = 0; i < v.size(); i++) {
21         for (int j = 0; j < v[i].size(); j++) {
22             cout << v[i][j] << ' ';
23         }puts("");
24     }
25 }

```

## queue

队列是一种先进先出（FIFO）的数据结构，元素按照其被添加的顺序进行处理。

## 常用函数

| 常用函数      |           |
|-----------|-----------|
| q.push(x) | 在队尾插入元素x  |
| q.pop(x)  | 删除队首元素    |
| q.size()  | 返回队列中元素个数 |
| q.empty() | 返回队列是否为空  |
| q.front() | 返回队首元素    |
| q.back()  | 返回队尾元素    |
|           |           |

## 定义及初始化

`queue<Type, Container>` // (<数据类型, 容器类型>)

初始化时必须要有数据类型，容器可省略，省略时则默认为[deque](#) 类型

```
1 //queue的定义 例
2 queue<int>q1; //定义一个储存数据类型为int的queue容器q1
3 queue<double>q2; //定义一个储存数据类型为double的queue容器q2
4 queue<string>q3; //定义一个储存数据类型为string的queue容器q3
5 queue<结构体类型>q4; //定义一个储存数据类型为结构体类型的queue容器q4
```

## 清空

queue没有clear()函数；可以通过以下方式清空队列

```
1 queue<int>q;
2 //方法一
3 q = queue<int>();
4 //方法二
5 queue<int>empty;
6 swap(q, empty);
7 //方法三
8 while(q.size()) q.pop();
```

## 元素访问

queue队列的元素只能访问队首元素或队尾元素，通过.pop()每次将队首元素弹出后遍历队列

```
1 queue<int>q;
2 for (int i = 0; i < 10;i++) {
3     q.push(i*10);
4 }
5 for (int i = 0; i < 5;i++) {
6     cout << q.front() << ' ';
7     q.pop();
8 }
```

## 优先队列

每次元素入队后会按一定次序排列元素，只能访问队首元素，默认大根堆

| 常用函数       |          |
|------------|----------|
| pq.push(x) | 向队列插入元素x |
| pq.pop()   | 弹出队首元素   |
| pq.top()   | 返回队首元素   |
| pq.size()  | 返回队列元素个数 |
| pq.empty() | 返回队列是否为空 |

```
priority_queue<结构类型> 队列名;
priority_queueq0;
priority_queue<int,vector,less >q1;    //大根堆，父节点>子节点
priority_queue<int, vector, greater> q2; //小根堆，父节点<子节点
```

```
1 //例 利用大根堆实现元素自动从大到小排列
2 #include <iostream>
3 #include <queue>
4 using namespace std;
5 priority_queue<int>q;
6 int main() {
7     for (int i = 0;i < 5;i++){
8         int x; cin >> x;
9         q.push(x);
10    }
11    while (q.size()){
```

```

12         cout << q.top();
13         q.pop();
14     }
15     return 0;
16 }

```

```

1  ///结构体重载<运算符
2  struct Edge{
3      int s,x,y;
4      bool operator < (const Edge &e)const {
5          return s > e.s;//小根堆，堆顶为最小值
6          //return s < e.s;//大根堆，堆顶为最大值
7      }
8  };
9
10 priority_queue<Edge>pq;

```

## deque

双端队列容器，速度慢，能通过下标访问元素，对队首/队尾操作较快

### 常用函数

| 常用函数         |                      |
|--------------|----------------------|
| front()      | 返回第一个元素的引用。          |
| back()       | 返回最后一个元素的引用。         |
| assign()     | 用新元素替换原有内容。          |
| push_back()  | 在序列的尾部添加一个元素。O(1)    |
| push_front() | 在序列的头部添加一个元素。O(1)    |
| pop_back()   | 移除容器尾部的元素。O(1)       |
| pop_front()  | 移除容器头部的元素。O(1)       |
| insert()     | 在指定的位置插入一个或多个元素。O(n) |
| erase()      | 移除一个元素或一段元素。O(n)     |

| 常用函数            |                                                              |
|-----------------|--------------------------------------------------------------|
| clear()         | 移出所有的元素，容器大小变为 0。                                            |
| swap()          | 交换两个容器的所有元素。                                                 |
| emplace()       | 在指定的位置直接生成一个元素。                                              |
| emplace_front() | 在容器头部生成一个元素。和 push_front() 的区别是，该函数直接在容器头部构造元素，省去了复制移动元素的过程。 |
| emplace_back()  | 在容器尾部生成一个元素。和 push_back() 的区别是，该函数直接在容器尾部构造元素，省去了复制移动元素的过程。  |

## 定义及初始化

```

1 //1) 创建一个没有任何元素的空 deque 容器:
2 deque<int>d1;
3
4 //2) 创建一个具有 10 个元素的 deque 容器，其中每个元素都采用对应类型的默认值:
5 deque<int>d2(10,-1); //int类型初始化为-1,第二个参数不填则默认为0
6
7 //3) 创建一个具有 10 个元素的 deque 容器，并为其初始化
8 deque<string>d3(10,"abc");
9
10 //4) 拷贝构建
11 deque<int> d4(d1);

```

## list

双向循环链表容器,不支持通过下标访问元素，插入删除时间复杂度都为O(1)

访问效率效率低 O(n)

//迭代器不可用NULL判空，如if(it == NULL)为非法判断

//可以将其指向end(),再利用end()判断

## 常用函数

| 常用函数               |                                               |
|--------------------|-----------------------------------------------|
| push_back/front(x) | 将元素x插入容器末尾 /队首 O(1)                           |
| pop_back/front()   | 删除末尾/队首元素 O(1)                                |
| insert(it,x)       | 在迭代器it前一个位置插入元素x O(1)                         |
| erase()            | 移除迭代器first(到last之间)的元素, 删除后迭代器失效              |
| size()             | 返回容器元素个数                                      |
| empty()            | 返回容器是否为空                                      |
| clear()            | 清空容器                                          |
| front()/back()     | 返回队首/末尾元素                                     |
| begin()            | 返回指向容器开始的迭代器                                  |
| end()              | 返回指向容器末尾的迭代器,解引用后为容器大小,next(end())又循环为begin() |

## 遍历链表

```
queueq;

1 //利用迭代器
2 for (list<int>::iterator it = q.begin(); it != q.end(); it++) {
3     cout << *it << " ";
4 }

1 //fou(auto& i:q)  c++11
2 for (int& i:l){
3     cout << i << ' ';
4 }
```

## 二维链表

```

1  #include <iostream>
2  #include <list>
3
4  void dfs(vector<list<int>>&q,int u){
5      for(auto& k:q[u]){
6          cout << k << endl;
7      }
8  }
9
10 int n;
11 vector<list<int>>q(n+1);
12 dfs(q,1);

```

## stack

栈，先进后出

### 常用函数

| 常用函数       |            |
|------------|------------|
| sk.push()  | 插入一个元素     |
| sk.pop()   | 弹出栈顶元素     |
| sk.top()   | 返回栈顶元素     |
| sk.size()  | 返回元素个数     |
| sk.empty() | 返回栈中元素是否为空 |

### 清空

由于stack没有clear()函数，可以手动清空，或者sk = stack();

### 例

```

1  #include <iostream>
2  #include <stack>
3  using namespace std;
4  stack<int>p;

```



```

5  int main() {
6      for (int i = 0; i < 5;i++) {
7          int x; cin >> x;//1 2 3 4 5
8          p.push(x);
9      }
10     for (int i = 0; i < 5;i++) {
11         cout << p.top();//54321
12         p.pop();
13     }
14     return 0;
15 }

```

## set

set是以rb\_tree为底层机构，因此有元素自动排序的特性。元素不能被改变

set 不可存在重复的数

## 常用函数

| setq               |                                                                        |
|--------------------|------------------------------------------------------------------------|
| insert()/emplace() | 插入一个数                                                                  |
| find()             | O(logn)查找一个数,返回这个数的迭代器,不存在则返回end()地址                                   |
| erase()            | 根据参数，删除一个数x/一个迭代器it/两个迭代器[it1,it2)之间的值                                 |
| empty()            | 判断容器是否为空                                                               |
| size()             | 返回当前容器元素个数                                                             |
| clear()            | 删除容器中所有元素                                                              |
| begin()            | 返回指向容器第一个元素的迭代器                                                        |
| end()              | 返回指向容器末尾元素的迭代器，解引用后为容器元素个数                                             |
| q.lower_bound(x)   | 返回第一个大于等于x的迭代器O(logn),不要使用lower_bound(last,end,x)会退化为O(N)              |
| q.upper_bound(x)   | 返回第一个大于x的迭代器                                                           |
| equal_range()      | 返回元素值为n的区间(pair类型) // *a.equal_range().first和 *a.equal_range()second之间 |

## 例

```
1  #include <iostream>
2  #include <set>
3  using namespace std;
4  set<int>p;
5  int main() {
6      for (int i = 0; i < 5;i++) {
7          int x; cin >> x;
8          p.insert(x);
9      }
10     /*cout << *p.lower_bound(2) << endl;
11     cout << *p.upper_bound(2) << endl;*/
12     cout << *p.equal_range(2).first;
13     cout << *p.equal_range(2).second;
14
15     for (auto& it= p.begin();it != p.end();it++) {//通过迭代器遍历
16         cout << *it << endl;/**号解引用
17     }
18     p.erase(p.begin(), p.end());
19
20     return 0;
21 }
```

## 重载

```
1  //重载为降序
2  struct cmp{
3      bool operator () (const int &e1,const int &e2)const{
4          return e1 > e2;
5      }
6  };
7
8  set<int,cmp>se;
```

```

1 //结构体重载
2 struct Edge{
3     int a,b;
4     bool operator < (auto &e2) const{
5         return a < e2.a; //升序
6         //return a > e2.a; //降序
7     }
8 };
9 set<Edge>se;

```

## multiset

可以存在重复的数

multiset中count(x)时间复杂度为 $O(k+\log(N))$ ，其中k为元素x出现的次数，N为集合大小

multiset.erase(x);会把所有的x删除，若只想要删一个，则应删除迭代器

## unordered\_set

```
#include <unordered_set>
```

与set唯一的区别就在于 unordered\_set 容器不会自行对存储的数据进行排序

## bitset

每位只占1bite空间，支持各种位运算如

~ | & ^ << >> [] == != 等

## 常用函数

| 常用函数    |                                              |
|---------|----------------------------------------------|
| size()  | 返回长度                                         |
| count() | 返回位值为1的数量                                    |
| set()   | 无参则将所有位设为1，一个参数则将下标为pos设为1，两个参数则将下标为pos的位设为k |

| 常用函数              |                                |
|-------------------|--------------------------------|
| reset()           | 无参将所有位设为0，一个参数则将下标为pos的位设为0    |
| flip()            | 无参则将所有位取反，一个参数则将下标为pos的位取反     |
| all()             | 是否所有位都是1                       |
| none()            | 是否所有位都是0                       |
| any()             | 是否至少有一个1                       |
| b.to_string()     | 返回为字符串型                        |
| b.to_ulong()      | 返回为无符号整型 to_ullong为长整型         |
| b._Find_first()   | 返回第一个1的位置，如果没有则返回b.size()      |
| b._Find_next(pos) | 返回pos后面第一个1的位置，如果没有则返回b.size() |
| << &   ^          | 支持 &   ^ 和移位操作                 |

## 初始化

```

1  bitset<31>b = n;           //int n 转二进制,b[0]~b[30]为二进制低位到高位,cout<<b;为高位到低位
2  bitset<N> bitset1;        // 创建一个长度为 N 的 bitset, 所有位默认初始化为 0
3  bitset<N> bitset2(value); // 使用二进制整数 value 初始化一个长度为 N 的 bitset //无符号整数
4  bitset<N> bitset3(string); // 使用二进制字符串 string 初始化一个长度为 N 的 bitset//只含'0'或'1' //bitset<N>b3(string("00110101"));
5  bitset<N> bitset4(bitset); // 使用另一个 bitset 初始化一个长度为 N 的 bitset

```

```

1  bitset<32>b;
2
3  int n = 127;//int b = 0b111111;
4  b = n;//使用整型
5
6  string s = "111111";
7  b = bitset<32>(s);//使用二进制字符串(构造函数)
8  //bitset<32>b("111111");

```

```

1  bitset<N>dp[N]; //运算效率比int[][]数组高
2
3  for(int i = 1; i <= n; i++){
4      int a,b; cin >> a >> b;
5      dp[a][b] = 1;
6  }
7  for(int i = n; i >= 1; i--){
8      dp[i] |= dp[i+c[i]];
9      ans = max(ans, dp[i].count());
10 }

```

## iomanip

### 常用函数

| 常用函数            |                    |
|-----------------|--------------------|
| fixed           | 固定以浮点数形式输出         |
| setprecision(n) | 保留n位小数             |
| setw(n)         | 设置位宽为n             |
| left right      | 左/右对齐              |
| setfill(c)      | 以字符c填充位宽空位         |
| dec oct hex     | 以10进制, 8进制, 16进制输出 |
| setbase(n)      | 以n进制输出 n只能为10,8,16 |

### setprecision

用于设置浮点数精度

//超过对应数据类型范围仍然会掉精度

```

1 double n = 1.145141919810;
2 cout <<fixed << setprecision(6) << n << endl;
3 //将a四舍五入到n位，不足n位则会自动添0（如果n为整型，n*1.0即可）
4
5 cout << 1.1 << endl; //会对后续其它浮点数输出造成影响
6 cout << defaultfloat << 1.1 << endl; //defaultfloat可以取该影响
7 cout << 1.1 << endl;

```

```

1 double a = 1.1234567;
2 a = (int)(1000.0 * a + 0.5) / 1000.0; //保留3位小数，四舍五入，根据需要增删0

```

```

1 //c语言自定义保留n位小数，四舍五入
2 #include<stdio.h>
3 int main(){
4     int n;
5     scanf("%d",&n);
6     printf("%.*f",n,123.666666);
7 }

```

## setw

设置 默认右对齐

```

1 int a = 1, b = 12, c = 123;
2 cout << setw(4) << a << endl;
3 cout << setw(4) << b << endl;
4 cout << setw(4) << c << endl;

```

## 对齐

left //输出左对齐  
right //输出右对齐

```

1 for (int i = 0; i < 5; i++){
2     cout << setw(5) << left << i;
3 }

```

## setfill

字符c

```
1 string s = "abcde";
2 cout << setfill('*') << setw(10) << s << endl;
```

## numeric

### 常用函数

| 常用函数                               |                                     |
|------------------------------------|-------------------------------------|
| accumulate(begin,end,value)        | 将区间内的元素与初始值相加，返回他们的和(返回值类型与value相同) |
| partial_sum(begin,end,res)         | 区间前缀和，并将结果写入res                     |
| adjacent_difference(begin,end,res) | 区间差分                                |
| iota(begin,end,value)              | 以value为初始值，批量递增赋值                   |

## accumulate

accumulate(first,last,value)

```
1 const int N = 1e9;
2 vector<int>a = {N,N,N,3,1,5,7,5,4};
3 long long sum = accumulate(a.begin(),a.end(),0LL);//需要注意溢出问题
```

## partial\_sum

求前缀和函数

partial\_sum(first,last,res) 区间求前缀和并将结果写入res

```

1 | vector<int>a = {2,6,8,3,1,5,7,5,4},b(3);
2 | partial_sum(a.begin(),a.end(),b.begin());//b = 2 8 16
3 | partial_sum(a.begin(),a.end(),a.begin());//原地求前缀和

```

## adjacent\_difference

差分函数，与前缀和写法类似

## iota

iota(begin,end,value) 将 [bg,ed) 中的元素依次赋值为 val,val+1,val+2,..., 复杂度 O(N)。

```

1 | //用于原地初始化并查集
2 | int a[100],n = 10;
3 | iota(a+1,a+n+1,1);//a = 0,1,2,3,4,5,6,7,8,9,10

```

## sstream

多次使用同一个stringstream对象，每次使用后都需要对其进行清空，既要清空其内容，又要清空其状态

## 定义及初始化

```

1 | stringstream ss;      //定义
2 | ss.clear();           //清空状态，仅仅清空标志位，并没有释放内存
3 | ss.str("");          //初始化为空串，清除stringstream内容
4 | ss.str(s);           //以字符串s初始化ss
5 | ss.str();            //转化为string类型，cout << ss.str() << endl;
6 | ss.fail();           //返回ss是否输入成功

```

```

1 | #include <iostream>
2 | #include <sstream>
3 | using namespace std;
4 |
5 | int main(){
6 |     stringstream ss;
7 |     string s = "111 222";
8 |     int a,b,c;
9 |     ss << s;

```



```

10     ss >> a >> b; //返回值为ss.fail(),ss已经为空,返回1
11
12     ss.clear();//需要清空ss的状态才能继续使用
13     ss << "333";
14     ss >> c;
15
16     cout << a << " " << b << " " << c << endl;
17 }

```

## 例

```

1 //例: 输入第一行一个数字N, 接下来N行数据, 每一行数据为个数不确定的整数。将每行的总和输出出来
2 #include <iostream>
3 #include <sstream>
4 using namespace std;
5
6 int main(){
7     int tt;cin >> tt;
8     stringstream ss;
9     getchar();//忽视第一个n后的回车
10    while(tt--){
11        string s;
12        getline(cin,s);//getline(cin,s,'\n');默认以回车作为结束符
13        int ans = 0,x = 0;
14
15        ss.clear();
16        ss.str(s); //或ss << s;
17
18        while(ss >> x){
19            ans += x;
20        }
21        cout << ans << endl;
22    }
23 }

```

```

1 //例: 分割单词
2 #include <iostream>
3 #include <sstream>
4 using namespace std;
5
6 int main(){
7     string s;
8     stringstream ss;
9     while(getline(cin,s)){//以EOF结束
10        ss.clear();
11        ss << s;
12        string word;
13        while(ss >> word){
14            cout << word << endl;

```

```

15     }
16     }
17 }

```

## tuple

#include

tuple (元组) 在c++11开始引用，是一个固定大小的**不同类型值的集合**，是泛化的std::pair。我们也可以把他当做一个通用的结构体来用，不需要创建结构体又获取结构体的特征，在某些情况下可以取代结构体使程序更简洁，直观。std::tuple理论上可以有无数个任意类型的成员变量，而std::pair只能是2个成员，因此在需要保存3个及以上的数据时就需要使用tuple元组了。

## 声明

```

1 tuple<int,string,char,bool> t;

```

## 元素访问

get<idx>(t) //访问元组t中的第idx个元素（从0开始）

```

1 //get<Idx>() 引用访问，其中idx不能用变量替代,如i,j,k等
2 cin >> get<0>(t);
3 cout << get<0>(t) << endl;

```

```

1 //c++17 结构化绑定访问
2 for_each(t+1,t+n+1,[&](auto &x){auto &[a,b,c,d] = x;cin >> a >> b >> c >> d;});
3 for(int i = 1;i <= n;i++){
4     auto &[num,name,sex,st] = t[i];
5     cout << name << ' ' << num << ' ' << sex << ' ' << st << endl;
6 }
7
8 for(int i = 1;i <= n;i++){
9     auto &[num,name,sex,st] = t[i];
10    cout << name << ' ' << num << ' ' << sex << ' ' << st << endl;
11 }

```

## tie

可以用于解包pair、tuple

引用参数为引用效果

```
1 int a = 10, b = 20, c = 30, d = 40;
2 tie(a, b, c, d) = make_tuple(b, a, d, c); // a = 20, b = 10, c = 40, d = 30;
3 // tie(a, b) = make_pair(b, a+b); // 可以用来递推求斐波那列
4 // tie(a, b) = pair{b, a+b};
5 // 性能不太好
6 pair<int, int> p = {333, 666};
7 tie(a, b) = p; // a = 333, b = 666;
```

可以用于为结构体引入字典序比较

```
1 struct Edge{
2     int n, k;
3     string s;
4     bool operator < (auto &e){
5         return tie(n, s, k) < tie(e.n, e.s, e.k);
6     }
7 };
8
9 int main(){
10     Edge e1 = {12, "abc", 56};
11     Edge e2 = {12, "abd", 56};
12     if(e1 < e2){cout << "YES";}
13     else cout << "NO";
14 }
```

---

## ctime

### clock\_t

```
1 // 计算程序运行某一段的耗时(ms)
2 // 例1
3 #include <iostream>
4 #include <ctime>
5 using namespace std;
```

```

6 clock_t start1, end1, start2, end2;
7 int main() {
8     start1 = clock();
9     for (int i = 0; i < 5e8; i++){
10         int b; b = i;
11     }
12     end1 = clock() - start1;
13     cout << end1 << "ms" << endl;
14
15     start2 = clock();
16     for (int i = 0; i < 1e9; i++){
17         int b; b = i;
18     }
19     end2 = clock();
20     cout << end2 - start2 << "ms" << endl;
21 }

```

```

1 //例2
2 #include <iostream>
3 using namespace std;
4 int main() {
5     int start1 = clock();
6     for (int i = 0; i < 1e7; i++);
7     int end1 = clock();
8     cout << end1 - start1 << "ms" << endl;
9
10    int start2 = clock();
11    for (int i = 0; i < 1e9; i++);
12    int end2 = clock();
13    cout << end2 - start2 << "ms" << endl;
14
15    return 0;
16 }

```

---

## 扩展库ext

[libstdc++: detail Directory Reference\(gnu.org\)](http://libstdc++.detail.Directory.Reference.gnu.org)

```

1  #include <bits/extc++.h>                //扩展库万能头
2
3  #include <ext/pb_ds/assoc_container.hpp> //关联容器
4  #include <ext/pb_ds/hash_policy.hpp>    //哈希表
5  #include <ext/pb_ds/tree_policy.hpp>    //平衡树
6  #include <ext/pb_ds/trie_policy.hpp>    //字典树
7  #include <ext/pb_ds/priority_queue.hpp> //堆
8  using namespace __gnu_pbds;

```

## 关于pb\_ds

pb\_ds 库全称 Policy-Based Data Structures。

pb\_ds 库封装了很多数据结构，比如哈希（Hash）表，平衡二叉树，字典树（Trie 树），堆（优先队列）等。

就像 `vector`、`set`、`map` 一样，其组件均符合 STL 的相关接口规范。部分（如优先队列）包含 STL 内对应组件的所有功能，但比 STL 功能更多。

pb\_ds 只在使用 libstdc++ 为标准库的编译器下可以用。

可以使用 `begin()` 和 `end()` 来获取 `iterator` 从而遍历

可以 `increase_key`、`decrease_key` 以及删除单个元素

## hash

```

1  #include <ext/pb_ds/assoc_container.hpp>
2  // #include <ext/pb_ds/hash_policy.hpp>
3  using namespace __gnu_pbds;

```

## 定义

```

1  gp_hash_table<string, bool> hs; //探测法（稍微快一点）
2  cc_hash_table<string, bool> hs; //拉链法

```

操作和map类似，支持[]和find()操作

## 例

```
1 //例：字符串哈希 https://www.luogu.com.cn/problem/U461211
2 #include <bits/stdc++.h>
3 #include <ext/pb_ds/assoc_container.hpp>
4 // #include <ext/pb_ds/hash_policy.hpp>
5 using namespace std;
6 using namespace __gnu_pbds;
7
8 int main(){
9     gp_hash_table<string,bool>hs;
10    int n,ans = 0;cin >> n;
11    while(n--){
12        string s;cin >> s;
13        if(hs.find(s) == hs.end()) hs[s] = 1,ans++;
14    }
15    cout << ans;
16 }
```

## tree

pb\_ds里的tree都是平衡树，一般使用 `rb_tree_tag` 红黑树，性能较好，元素有序不可重

```
1 #include <ext/pb_ds/assoc_container.hpp>
2 // #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
```

## 定义

`tree<Key, Mapped, Cmp_Fn, Tree_Tag, Node_Update>tr`

`Key`：储存的**元素类型**，如int,pair,struct等,并配合lower\_bound和upper\_bound成员函数查找

`Mapped`：映射规则：[集合][类似set](则此处填**null\_type**，低版本g++填null\_mapped\_type)，[带值集合][类似map](此处填类似于map<Key,Value>的**Value类型**)

`Cmp_Fn`：关键字的**比较函数**，例如less<Key>从小到大

`Tree_Tag`：使用何种**底层数据结构类型**，默认是红黑树rb\_tree\_tag

`Node_Update`：用于更新**节点的策略**，默认null\_node\_update，若要使用求排名order\_of\_key和找k小值find\_by\_order方法，需要使用tree\_order\_statistics\_node\_update

```
1 tree<int,null_type,less<int>,rb_tree_tag,tree_order_statistics_node_update>rb;
```

## 常用函数

| 常用函数                 |                               |
|----------------------|-------------------------------|
| rb.insert(x);        | 插入x                           |
| rb.erase(x);         | 删除x                           |
| rb.lower_bound(x);   | 返回第一个大于等于x的迭代器                |
| rb.upper_bound(x);   | 返回第一个大于x的迭代器                  |
| rb.order_of_key(x);  | 求x的排名(下标从0开始)                 |
| rb.find_by_order(x); | 求第x小的值(下标从0开始)                |
| rb.join(rb2);        | 将rb2合并进rb, 前提是两棵树类型一样且没有重复元素  |
| rb.split(x,rb2);     | 分裂, key小于等于x的元素属于rb, 其余的属于rb2 |

## 例

```
1 //例: 普通平衡树 https://www.luogu.com.cn/problem/P3369
2 #include <bits/stdc++.h>
3 #include <ext/pb_ds/assoc_container.hpp>
4 // #include <ext/pb_ds/tree_policy.hpp>
5 using namespace std;
6 using namespace __gnu_pbds;
7
8 int main(){
9
10     tree<int, null_type, less_equal<int>, rb_tree_tag, tree_order_statistics_node_update> rb;
11     // less_equal 或 使用pair<int, int>+umap时间戳, 使得能存在相同key
12     int n; cin >> n;
13     for(int i = 1; i <= n; i++){
14         int op, x; cin >> op >> x;
15         if(op == 1) rb.insert(x); // 插入x(有重复x)
16         if(op == 2) rb.erase(rb.upper_bound(x)); // 删除x(只删除一个)
17         if(op == 3) cout << rb.order_of_key(x)+1 << '\n'; // 查询x的排名(树中下标从0开始, 结果要+1)
18         if(op == 4) cout << *rb.find_by_order(x-1) << '\n'; // 查询排名为x的值(同上, 查询时要-1)
```

```

18         if(op == 5) cout << * (--rb.upper_bound(x)) << '\n'; //查询x的前驱(小于x的最
    大值)
19         if(op == 6) cout << *rb.lower_bound(x) << '\n'; //查询x的后继(大于x的最小值)
20     }
21 }

```

```

1  #include <bits/stdc++.h>
2  #include <ext/pb_ds/assoc_container.hpp>
3  // #include <ext/pb_ds/tree_policy.hpp>
4  using namespace std;
5  using namespace __gnu_pbds;
6  using pii = pair<int, int>;
7  unordered_map<int, int> mp;
8
9  int main(){
10
11     tree<pii, null_type, less<pii>, rb_tree_tag, tree_order_statistics_node_update> rb;
12
13     int n; cin >> n;
14     for(int i = 1; i <= n; i++){
15         int op, x; cin >> op >> x;
16         if(op == 1) rb.insert({x, mp[x]++}); //插入x(有重复x, 先插入再++)
17         if(op == 2) rb.erase(rb.lower_bound({x, --mp[x]})); //删除x(只删除一个, 先--再删除)
18         if(op == 3) cout << rb.order_of_key({x, 0}) + 1 << '\n'; //查询x的排名(树中下标从0开始, 结果要+1)
19         if(op == 4) cout << rb.find_by_order(x - 1) ->first << '\n'; //查询排名为x的值(同上, 查询时要-1)
20         if(op == 5) cout << (--rb.lower_bound({x, 0})) ->first << '\n'; //查询x的前驱(小于x的最大值)
21         if(op == 6) cout << rb.upper_bound({x, mp[x]}) ->first << '\n'; //查询x的后继(大于x的最小值)
22     }
23 }

```

## 堆

```

1  #include <ext/pb_ds/priority_queue.hpp>
2  using namespace __gnu_pbds;

```



定义

```
1 __gnu_pbds::priority_queue<int,greater<int>> pq;  
2 //要注明命名空间，防止与std冲突  
3 //第二个参数不写则默认大根堆  
4 //第三个tag参数不写默认为配对堆pairing_heap_tag，性能较好
```

| 不同tag性能比较                       | push                                | pop                                 | modify                              | erase                               | Join              |
|---------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------|
| pairing_heap_tag<br>配对堆         | $O(1)$                              | 最坏 $\Theta(n)$ 均摊 $\Theta(\log(n))$ | 最坏 $\Theta(n)$ 均摊 $\Theta(\log(n))$ | 最坏 $\Theta(n)$ 均摊 $\Theta(\log(n))$ | $O(1)$            |
| binary_heap_tag<br>二叉堆          | 最坏 $\Theta(n)$ 均摊 $\Theta(\log(n))$ | 最坏 $\Theta(n)$ 均摊 $\Theta(\log(n))$ | $\Theta(n)$                         | $\Theta(n)$                         | $\Theta(n)$       |
| binomial_heap_tag<br>二项堆        | 最坏 $\Theta(\log(n))$ 均摊 $O(1)$      | $\Theta(\log(n))$                   | $\Theta(\log(n))$                   | $\Theta(\log(n))$                   | $\Theta(\log(n))$ |
| rc_binomial_heap_tag<br>冗余计数二项堆 | $O(1)$                              | $\Theta(\log(n))$                   | $\Theta(\log(n))$                   | $\Theta(\log(n))$                   | $\Theta(\log(n))$ |
| thin_heap_tag<br>简化的斐波那契堆       | $O(1)$                              | 最坏 $\Theta(n)$ 均摊 $\Theta(\log(n))$ | 最坏 $\Theta(\log(n))$ 均摊 $O(1)$      | 最坏 $\Theta(n)$ 均摊 $\Theta(\log(n))$ | $\Theta(n)$       |

常用函数

| 常用函数             |                                                                              |
|------------------|------------------------------------------------------------------------------|
| pq.push(x);      | 插入x,会返回一个指向x所在位置的<br>__gnu_pbds::priority_queue<...>::point_iterator 的点类型迭代器 |
| pq.pop();        | 弹出堆顶元素                                                                       |
| pq.top();        | 返回堆顶元素                                                                       |
| pq.modify(it,x); | 将点迭代器it位置的键值改为x，会返回一个点迭代器，并对底层存储结构排序                                         |
| pq.erase(it);    | 把点迭代器it所在位置的键值从堆中擦除                                                          |
| pq.join(pq2);    | 把堆pq2并入到堆pq，并清空pq2。                                                          |
| pq.size();       | 返回元素个数                                                                       |
| pq.empty();      | 返回是否为空                                                                       |

对于不同的tag参数，其底层实现并不相同，点迭代器的失效条件也不一样。可能在pop或erase后，点迭代器依然不会失效。

不支持直接获取堆顶的点迭代器，在push或modify操作时获取。对迭代器begin()解引用后也并不等于top()。

无法通过点迭代器合并堆，也无法仅凭点迭代器找到所属堆进行合并，必须显示持有目标堆的引用。

## 例

[P3377 【模板】左偏树/可并堆 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P3377)

```
1  #include <bits/stdc++.h>
2  #include <ext/pb_ds/priority_queue.hpp>
3  using namespace std;
4  using namespace __gnu_pbds;
5  using pii = pair<int,int>;
6  const int N = 100005;
7  int n,q,p[N];
8  bool st[N];
9  __gnu_pbds::priority_queue<pii,greater<pii>> pq[N];
10
11 int find(int x){
12     if(p[x] != x) p[x] = find(p[x]);
13     return p[x];
14 }
15
16 int main(){
17     cin >> n >> q;
18     for(int i = 1;i <= n;i++){ p[i] = i; }
19     for(int i = 1;i <= n;i++){
20         int x;cin >> x;
21         pq[i].push({x,i});
22     }
23
24     while(q--){
25         int op;cin >> op;
26         if(op == 1){//将第a个数和第b个数所在堆合并, 若已被删除或已经在同一个堆则无视该操作
27             int a,b;cin >> a >> b;
28             int pa = find(a),pb = find(b);
29             if(st[a] || st[b] || pa == pb) continue;
30             pq[pa].join(pq[pb]);
31             p[pb] = pa;
32         }
33         else{//输出第a个数所在堆的最小数,然后将最小数删除(若有多个最小数,优先删除先输入的)。
34             //若第a个数已经被删除,输出-1,并无视删除操作。
35             int a;cin >> a;
36             if(st[a]) {cout << -1 << '\n';continue;}
37
38             int pa = find(a);
39             st[pq[pa].top().second] = 1;
```

```

40         cout << pq[pa].top().first << '\n';
41         pq[pa].pop();
42     }
43 }
44 }

```

#### P11266 【模板】可并堆 2 - 洛谷 (luogu.com.cn)

维护序列 $a[n]$ , 和 $n$ 和集合 $s[n]$ , 初始时 $s[i] = i$

- $0 \ x \ y$ : 表示将元素  $y$  从集合  $S_x$  中删去。保证此时元素  $y$  在集合  $S_x$  中。
- $1 \ x$ : 表示询问  $\min_{i \in S_x} a_i$ , 保证此时集合  $S_x$  非空。
- $2 \ x \ y$ : 将集合  $S_y$  中并入  $S_x$  并清空集合  $S_y$ 。保证此时集合  $S_x, S_y$  均非空, 且此次操作后不会再出现涉及集合  $S_y$  的操作。
- $3 \ x \ y \ z$ : 表示将  $a_y$  赋值为  $z$ 。保证此时元素  $y$  在集合  $S_x$  中, 且  $z < a_y$ 。

```

1  #include <bits/stdc++.h>
2  #include <ext/pb_ds/priority_queue.hpp>
3  using namespace std;
4  const int N = 1000006;
5  __gnu_pbds::priority_queue<int, greater<int>> pq[N];
6  __gnu_pbds::priority_queue<int, greater<int>>::point_iterator it[N];
7
8  int main() {
9      int n, q; cin >> n >> q;
10     for(int i = 1; i <= n; i++){
11         int x; cin >> x;
12         it[i] = pq[i].push(x);
13     }
14     while(q--){
15         int op, x, y, z; cin >> op;
16         if(op == 0){
17             cin >> x >> y;
18             pq[x].erase(it[y]);
19         }
20         if(op == 1){
21             cin >> x;
22             cout << pq[x].top() << '\n';
23         }
24         if(op == 2){
25             cin >> x >> y;
26             pq[x].join(pq[y]);
27         }
28         if(op == 3){
29             cin >> x >> y >> z;
30             pq[x].modify(it[y], z);
31         }
32     }
33 }

```

# trie

字典树，功能较少，不能满足大多数需求

```
1 | #include <ext/pb_ds/assoc_container.hpp>
2 | //#include <ext/pb_ds/trie_policy.hpp>
```

## 定义

```
1 | trie<string,null_type,trie_string_access_traits<>,pat_trie_tag,trie_prefix_search  
   | _node_update> tr;
```

## 常用函数

| 常用函数                            |                                                            |
|---------------------------------|------------------------------------------------------------|
| tr.insert(s);                   | 插入字符串s                                                     |
| tr.erase(s);                    | 删除字符串s                                                     |
| tr.join(tr2);                   | 将tr2并入tr                                                   |
| auto p =<br>tr.prefix_range(s); | 查询以s为前缀的字符串有哪些，p为两个迭代器pair<tr::iterator ,<br>tr::iterator> |

## 例

```
1 | //https://www.luogu.com.cn/problem/P8306
2 | //求 S1~SN 中 T 是多少个字符串的前缀
3 | #include <bits/stdc++.h>
4 | #include <ext/pb_ds/assoc_container.hpp>
5 | #include <ext/pb_ds/trie_policy.hpp>
6 | using namespace std;
7 | using namespace __gnu_pbds;
8 |
9 | void sol(){
```

```

10     trie<string,null_type,trie_string_access_traits<>,pat_trie_tag,trie_prefix_searc
        h_node_update> tr;
11     int n,q;cin >> n >> q;
12     for(int i = 1;i <= n;i++){
13         string s;cin >> s;
14         tr.insert(s);
15     }
16     while(q--){
17         string s;cin >> s;
18         auto p = tr.prefix_range(s);
19         int ans = 0;
20         for(auto it = p.first;it != p.second;it++){//时间复杂度为O(N*ans),并不能通
过本题
21             //cout << *it << endl;
22             ans++;
23         }
24         cout << ans << '\n';
25     }
26 }
27
28 int main(){
29     int T;cin >> T;
30     while(T--){sol();}
31 }

```

/

## rope

STL 中的 `rope` 也起到块状链表的作用，它采用可持久化平衡树实现，可完成随机访问和插入、删除元素的操作，可以在  $O(1)$  的时间拷贝历史版本

由于 `rope` 并不是真正的用块状链表来实现，所以它的时间复杂度并不等同于块状链表，而是相当于可持久化平衡树的复杂度（即  $O(\log n)$ ），但常数较大，空间需求较大，可以维护的东西较少

```

1 #include <ext/rope>
2 using namespace __gnu_cxx;

```

# 定义

与vector等容器类似，不可直接赋值，应用push\_back、replace、insert等修改元素

```
1 | rope<int>r;
```

## 常用函数

| 常用函数                                                                       |                                                                                     |
|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| r.push_back(x)<br>r.append(x)                                              | 在末尾添加元素x。空间复杂度O(log n)， 时间复杂度O(1)                                                   |
| r.pop_back()                                                               | 删除末尾元素。时间复杂度O(log n)， 时间复杂度O(1)                                                     |
| r.insert(pos,x)                                                            | 在pos位置前插入元素x， 时间复杂度O(log n + len)， 空间复杂度O(len)， len 为插入内容大小                         |
| r.erase(pos,len)                                                           | 删除pos位置起， len个元素， 时间复杂度O(log n + len)                                               |
| r.at(x) r[x]                                                               | 访问a的第x个元素， 时间复杂度O(log n)                                                            |
| r.size() r.length()                                                        | 获取r的大小                                                                              |
| lower_bound(begin,end,x)                                                   | 二分查找 $O(\log^2 n)$                                                                  |
| r.substr(begin,end)                                                        | 截取子段 [ begin , end )， 时间复杂O(log n + len)， 空间复杂度O(len)                               |
| r.replace(i, len, r2)<br>r.replace(i, len, s)<br>r.replace(i, len, cnt, c) | 将i位置开始长度为len个元素删掉， 替换为r2/c风格字符串s/cnt个字符c。<br>时间复杂度为 $O(\log n) + m$ ， 其中m为新插入内容的长度。 |

## 例

```
1 | //数列分块入门6(元素插入和删除)  https://loj.ac/p/6282
2 | #include <iostream>
3 | #include <ext/rope>
4 | using namespace std;
5 |
6 | int main(){
7 |     __gnu_cxx::rope<int>a;
8 |     int n;cin >> n;
9 |     for(int i = 1;i <= n;i++){
10 |         int x;cin >> x;
11 |         a.push_back(x);
12 |     }
13 |     while(n--){
14 |         int op,l,r,c;cin >> op >> l >> r >> c;
```

```

15         if(op == 0){//在第l个数字前插入r
16             a.insert(l-1,r);
17         }
18         else{//询问第r个数字的值
19             cout << a[r-1] << '\n';
20         }
21     }
22 }

```

```

1 //文艺平衡树(区间翻转) https://www.luogu.com.cn/problem/P3391
2 #include <bits/stdc++.h>
3 #include <ext/rope>
4 using namespace std;
5 using namespace __gnu_cxx;
6 int n,m;
7 rope<int>a,b;
8
9 rope<int> sub(rope<int>&v,int l,int r){
10     return v.substr(v.begin()+l,v.begin()+r);
11 }
12
13 int main(){
14     cin >> n >> m;
15     for(int i = 1;i <= n;i++){
16         a.push_back(i); b.push_back(n-i+1);
17     }
18     while(m--){
19         int l,r;cin >> l >> r;//将区间l~r翻转
20         if(l >= r) continue;
21         auto tmp = sub(a,l-1,r);
22         a = sub(a,0,l-1) + sub(b,n-r,n-l+1) + sub(a,r,n);
23         b = sub(b,0,n-r) + tmp + sub(b,n-l+1,n);
24     }
25     for(int i = 0;i < n;i++){ cout << a[i] << ' '; }
26 }

```

```

1 //高级打字机(数据可持久化) https://www.luogu.com.cn/problem/P1383
2 #include <bits/stdc++.h>
3 #include <ext/rope>
4 using namespace std;
5 using namespace __gnu_cxx;
6 const int N = 100005;
7 rope<char>a[N],now;//a[]存历史版本,now为当前版本
8 int cnt;
9
10 int main(){
11     int q;cin >> q;

```

```

12     while(q--){
13         char op;cin >> op;
14         if(op == 'T'){ //在文章末尾添加一个小写字母x
15             char x;cin >> x;
16             now.append(x);
17             a[++cnt] = now;
18         }
19         else if(op == 'U'){ //撤销最后x次修改操作
20             int x;cin >> x;
21             now = a[cnt-x];
22             a[++cnt] = now;
23         }
24         else{
25             int x;cin >> x; //输出当前文章中第x个字母(本操作不算修改操作)
26             cout << now[--x] << '\n';
27         }
28     }
29 }

```



---



